Groundwork Project Documentation

Product overview, competitive positioning and technical stack

Version 1.0 · 20 October 2026

Groundwork is a browser based design and project workspace for interior designers, architects and site engineers. It combines a public architecture studio website with a private application for creating spatial designs, reviewing technical studies and managing project information. This document gives product stakeholders and developers a shared explanation of what the application does, how it is positioned and how it is built.

1 What Groundwork is

Purpose and intended users

Groundwork helps a design team move from an initial room or site concept to a saved, reviewable project. Users can work with dimensions, furniture, finishes and site layouts in a 3D browser interface, then retain revisions, exchange files and coordinate with other people who have permission to access the project.

Interior designers can explore room layouts and material choices. Architects can develop spatial and community concepts and prepare model handoffs. Site engineers can organize site context, review bounded infrastructure and engineering studies, and retain the inputs and outputs behind those studies. Project owners and reviewers can inspect shared designs without receiving unrestricted editing rights.

Groundwork has two connected experiences. The **public studio** presents the brand, services, illustrative projects and design process. The **authenticated workspace** holds account and project tools. Both are built together and served from the same application origin.

Main capabilities

Area	What users can do
Studio website	Explore a scroll driven Three.js point-scan city, services layouts, an eight-item project carousel and a staged 3D process scene. Motion controls, reduced motion and WebGL fallbacks support different devices and preferences.
Account access	Register, sign in, use email login codes, enable authenticator based TOTP MFA, manage settings and review account activity.
Project library	Create and organize projects, use templates, duplicate or archive work, and reopen saved designs and revisions.

Area	What users can do
Interior workspace	Model rooms, place and transform furniture, change finishes and colors, work with walls and floors, and inspect orbit or walkthrough views. An imported room photograph can serve as a modeling backdrop.
Site planning	Develop community and infrastructure concepts, use optional Google Maps site context and review planning geometry and quantities.
Technical workspaces	Save geometry, engineering and exchange studies alongside project records, with source revisions and related artifacts.
Coordination	Assign project access, exchange comments and issues, inspect snapshots, receive change notifications and create expiring read-only share links.
Handoff	Export DXF drawings, OBJ and MTL scenes, GLB models and CSV quantities. Additional workbenches cover declared IFC, IDS, BCF, survey and civil exchange subsets.

Typical project workflow

1. Open the studio and enter the application through sign-in or registration.
2. Create a room, community or infrastructure project and establish dimensions and site context.
3. Develop the design in the browser, then save it to the project record.
4. Review geometry, exchange or engineering studies where they apply. Retain the study inputs, source revision and output artifacts.
5. Invite permitted collaborators or issue a read-only share link. Resolve stale save conflicts before replacing newer work.
6. Export models, drawings or quantities for review and further work in specialist tools.

Scope and product boundaries

The core product is an integrated planning and coordination environment. Engineering modules implement bounded structural and MEP calculations, including selected India and US standards checks. Their applicability and input limits matter; they do not establish complete code compliance or independently certified engineering results. IFC and civil tools support stated subsets rather than unrestricted round trips through every authoring system.

A room photograph is a backdrop for modeling, rather than an automatic, dimensionally accurate 3D reconstruction. The public portfolio contains illustrative concept content with credited third-party photography; it should be replaced with verified studio projects before commercial publication. The newsletter requires a real subscription backend before it can accept signups.

2 What differentiates Groundwork

A shared workspace across related design tasks

Groundwork brings room interiors, community and infrastructure concepts, geometry review and bounded technical studies into one application. Shared account and project records let a team move between these tasks without creating a separate project identity for each tool. This integration is the central product emphasis.

Technical results retain their project context

Engineering, geometry and exchange workspaces can retain saved revisions, original artifacts and links to source design revisions. Users can distinguish a result generated for an older design from one linked to the current project state. This makes the relationship between a design and its supporting study visible during review.

File handoff is part of the workflow

DXF, OBJ and MTL, GLB and CSV outputs help users continue work outside Groundwork. Bounded IFC, IDS, BCF and civil exchange tools extend that handoff. Unsupported representations and approximation limits must remain visible so a downstream specialist can decide what to reuse or rebuild.

Collaboration includes conflict and recovery decisions

The server checks access and revision numbers before accepting project changes. A stale write returns a conflict instead of silently replacing newer work. Server-Sent Events distribute project metadata updates, while comments, issues and shares support review. Browser drafts and recovery exports help preserve interrupted work; this is bounded recovery and synchronization, rather than unrestricted simultaneous editing of every model element.

The studio identity continues into the application

The public site and authentication pages share an architectural point-scan visual language, an ivory, ink and copper palette, and locally hosted typography. The public scroll narrative introduces the studio; the private workspace supports practical design work. Both use a common build and deployment structure.

Position relative to representative competitors

These distinctions describe Groundwork's implementation and intended workflow. They are positioning judgments, not proof that its individual features are exclusive or that it outperforms competitors. The comparison covers representative product roles rather than every feature or subscription plan.

Product	Officially documented focus	Groundwork positioning
SketchUp for Web	Browser 3D modeling, Trimble Connect file management and collaboration tools.	Groundwork emphasizes shared project records spanning interiors, site concepts and bounded technical workspaces. Browser modeling and collaboration alone are not exclusive advantages.
Autodesk Revit	BIM design and documentation with model relationships, drawings and schedules.	Groundwork emphasizes browser access and concept-to-study coordination. Its bounded BIM and engineering modules should be assessed alongside specialist authoring tools; equivalence to Revit is not established.
Foyr Neo	Interior space planning, furniture placement, 3D visualization and cloud rendering.	Groundwork combines interior design with broader site and technical workspaces. Its optional AI image adapter is prompt based; faithful geometry-preserving rendering and parity with Foyr's catalog are not established.

Sources: SketchUp for Web documentation, Revit Fundamentals, and Foyr Neo product documentation, accessed 20 October 2026.

The strongest defensible description is **a browser design workspace that keeps spatial concepts, technical studies and project coordination together**. Claims about lower cost, superior numerical accuracy, faster performance or stronger security require separate evidence.

3 Technical architecture

Application structure

The repository is an npm workspace monorepo with a `client` and a `server`. Vite produces two HTML entry points. `/` serves the vanilla JavaScript studio document; application routes such as `/login`, `/dashboard` and `/editor/:id` serve the React document. React Router manages navigation inside the authenticated application.

The Express backend exposes `/api` endpoints and serves the built frontend in the production recipe. During development, Vite proxies API requests to the backend. The browser communicates with the backend rather than connecting directly to the project database.

```
Browser
  Public studio       HTML + modular CSS + JavaScript + Three.js
  Private workspace   React + TypeScript + Three.js
          |
          | same-origin HTTP API and Server-Sent Events
          v
Node.js and Express authority
  Authentication, permissions, validation, encryption and project services
          |
          | pg connection pool with certificate-validated TLS
          v
Supabase PostgreSQL
  Private groundwork schema and restricted backend runtime role

Local browser storage   IndexedDB drafts and bounded public shell cache
Optional services       Maps, mail, payments, AI, OIDC and Redis REST
```

Database and storage design

The current cloud database is **Supabase PostgreSQL 17**, in the US East North Virginia region. It contains a private `groundwork` schema with 37 application tables plus a migration ledger. Connections use the IPv4 session pooler with certificate validation. A restricted runtime role performs application operations; migrations use a separate elevated process.

Row Level Security is enabled, and the private tables are not exposed to anonymous or ordinary Supabase browser roles. The backend runtime is trusted to access application records, so Express authorization checks remain responsible for user and organization isolation. This is a server mediated database design, rather than browser access through per-user Supabase Auth policies.

SQLite remains an explicit local and test fallback through Node's built-in `node:sqlite` module. The shared asynchronous adapter exposes prepared queries, transactions and savepoints. PostgreSQL uses a connection pool, transaction ownership and commit hooks so notifications follow committed changes. Current write serialization favors correctness; large-scale write throughput still requires measurement and tuning.

Project payloads and supported private artifacts are encrypted by the server before persistence. Browser drafts and recovery ZIP files can contain plaintext user data. The server encryption claim therefore does not extend to every copy of a design or to all database metadata.

4 Frontend and visualization stack

Versions below are the resolved versions in `package-lock.json` for the relevant workspace on 20 October 2026. They describe this application, not the newest available releases. Transitive dependencies are retained in the lockfile rather than repeated here.

Languages and production dependencies

Technology	Version	Role
HTML5 and modern CSS	Web platform	Studio markup, responsive layout, custom properties, fluid sizing, component styles and motion preferences.
JavaScript ES modules	Web platform	Public studio behavior, scroll stages, gallery and Three.js scene lifecycle.
TypeScript	5.7.3	Typed application, backend, shared data contracts and technical modules.
React	18.3.1	Authenticated application components and stateful user interfaces.
React DOM	18.3.1	React rendering in the browser.
React Router DOM	7.18.4	Application routes, protected screens and share-link navigation.
Three.js	0.172.0	WebGL scenes, cameras, geometry, materials, interaction and model export support.
JSZip	3.10.2	ZIP packages for model handoff and recovery workflows.

Styling and frontend build tools

Tool	Version	Role
Vite	8.3.4	Development server and production build with separate studio and workspace entries.
React Vite plugin	6.1.2	React build integration and development refresh.
Tailwind CSS	4.3.3	Utility styling in the authenticated application.
Tailwind Vite plugin	4.3.3	Tailwind build integration.
Modular studio CSS	In repository	Tokens and styles divided by page area. Central tokens live in `client/studio/styles/tokens.css`.
DM Sans and Instrument Serif	Local font assets	Studio and authentication typography, with font licenses recorded alongside assets.

The application invokes Three.js directly; it does not depend on React Three Fiber. Studio animation is implemented in repository code rather than an installed GSAP or smooth-scroll package. The client workspace resolves Vite 8.3.4 even though the lockfile also retains a root-level Vite 6 dependency for another tool.

Browser platform features

WebGL supports the 3D viewports. Animation frames, intersection and resize observers coordinate rendering and layout. Web Workers isolate selected parsing and computational tasks. IndexedDB stores local drafts and recovery state. Service Workers and Cache Storage provide a bounded, build-fingerprinted public application shell; they do not cache private API responses as a general offline database.

Fetch handles HTTP requests and streaming HTTP supports event delivery. File and Blob APIs support imports and downloads. Camera and media APIs support permitted capture and presentation features where available. Browser and device support still determine availability; WebXR controls do not establish verified compatibility with every headset or phone.

5 Backend and security stack

Runtime and production dependencies

Technology	Version	Role
Node.js	Minimum 22.12.0	Server runtime; Node 22 is selected in deployment recipes. Supplies crypto, HTTP fetch, file, stream, worker and SQLite APIs.
Express	4.22.3	HTTP application, routes, middleware and frontend serving.
pg	8.23.1	PostgreSQL connection pooling, parameterized queries and transactions.
Zod	3.25.76	Validation of API input and structured provider output.
cookie-parser	1.4.7	Cookie parsing for server session flows.
express-rate-limit	7.5.1	Request throttling in addition to persistent account and provider limits.
Helmet	8.3.0	Security response headers and Content Security Policy configuration.
Multer	2.4.0	Bounded multipart upload parsing with application validation.

Technology	Version	Role
Nodemailer	10.0.16	SMTP mail transport, including Gmail with a Google app password.
qrcode	1.5.4	Authenticator enrollment QR images.
dotenv	16.6.1	Private server environment configuration in local workflows.

Authentication and data protection

Authentication belongs to the Express application. It uses random opaque session tokens stored as hashes in the database; this is not a browser JWT session design. Production session cookies use HttpOnly, Secure and SameSite settings. Credential changes and revocation invalidate affected sessions. Email login codes and optional authenticator based TOTP are implemented with expiry, attempt limits and replay controls.

Node's built-in crypto module supplies scrypt password hashing with random per-user salts, AES-256-GCM encryption and HKDF key derivation. The server holds the master key, so this is application-layer encryption at rest rather than end-to-end encryption against the server operator. The matching key must be preserved with backups and releases.

Server middleware and services enforce CSRF protection, permission checks, upload and payload limits, account lockout controls and provider resource budgets. Audit records track supported account and project activity. These controls are implemented in code; they are not a security certification or a guarantee that no vulnerability can exist. Operational alerting, retention and incident response need deployment configuration.

6 Integrations and custom modules

External service integrations

Service	Implementation and current role
Supabase	Active hosted PostgreSQL database. Access uses `pg`; Supabase Auth, Storage, Realtime and Edge Functions are not the application's replacement services.
Gmail SMTP	Current outgoing mail configuration. SMTP authentication was verified; successful connection alone does not prove inbox delivery. Other SMTP providers and a Resend HTTP adapter are also supported in code.
Google Maps JavaScript API	Optional site lookup and map context. Requires an owner-provided browser key with suitable restrictions.

Service	Implementation and current role
OpenAI compatible API	Optional server-side design assistant. Returns a validated proposal for supported actions; the plan endpoint does not directly mutate the project. Requires provider configuration.
Google Gemini and Imagen	Optional prompt-based image-generation adapter with project revision and source artifact provenance. The current adapter sends the prompt rather than the source image to the provider. Live API compatibility requires provider testing.
Razorpay	Optional live payment-link and verification flow, including supported organization seat terms. Requires merchant credentials and signed webhook configuration.
PayPal	Optional personal checkout and capture flow. Requires merchant configuration; it is not the organization seat billing provider.
Redis REST	Optional shared coordination events and presence transport for configured deployments. It is accessed through HTTP rather than an installed Redis client library.
OIDC identity providers	Optional organization sign-in integration with an allowlisted provider configuration. SAML requires an OIDC broker rather than a native SAML adapter.
External native tools	Fixed conversion and analysis adapters. Execution is disabled by default and unavailable in the current Windows runtime; compatible tools and isolated Linux operation require separate provisioning and acceptance.

Provider integrations use existing HTTP or SMTP facilities; the presence of an adapter does not mean a paid service is provisioned or active. Demo render output is a procedural test image and should not be presented as an AI render.

Modules developed inside the repository

Groundwork includes its own TypeScript modules for room and site geometry, catalog data, design merging, revision handling, quantities, DXF and OBJ output, BIM and IFC subsets, IDS validation, BCF coordination, survey import, terrain and civil alignments. Geometry streaming, triangle clash checks and progressive path tracing also live in repository code.

Engineering workbenches contain bounded structural, electrical, hydraulic and HVAC calculations, including linear frame models and selected standards checks. These are application modules, not installed commercial solvers. Full IFC coverage, comprehensive national-code design, validated production drawing scale and arbitrary proprietary model conversion are outside the established capability. Native tool contracts do not establish a working native execution service.

7 Development testing and deployment tools

Development dependency inventory

All directly declared development dependencies are accounted for below, including type packages that do not ship as browser or server features.

Dependency	Resolved version	Role
TypeScript	5.7.3	Client and server compilation and type checks.
tsx	4.23.15	TypeScript development and test execution.
concurrently	9.2.5	Starts the client and server development commands together.
Supabase CLI	2.121.0	Supabase development and database tooling.
PGlite	0.5.10	Local PostgreSQL integration test environment.
Vite and React plugin	8.3.4 and 6.1.2	Frontend build tools described in Section 4.
Tailwind and Vite plugin	4.3.3 and 4.3.3	Frontend CSS build tools described in Section 4.
React and React DOM types	18.3.31 and 18.3.7	Type definitions for UI components and rendering.
Three.js types	0.172.0	Type definitions for Three.js.
Google Maps types	3.66.5	Browser map API type definitions.
Node.js types	22.20.5	Runtime type definitions.
Express types	4.17.25	HTTP framework type definitions.
Cookie parser types	1.4.10	Cookie middleware type definitions.
Multer types	2.3.0	Upload middleware type definitions.
Nodemailer types	8.0.2	Mail transport type definitions.
pg types	8.23.1	PostgreSQL client type definitions.
QR code types	1.5.6	QR generation type definitions.

Verification and delivery

The Node built-in test runner executes server, browser-storage security and studio unit checks. PGlite supports PostgreSQL flow checks without a hosted test database. Browser smoke checks have also used Playwright as a QA tool; Playwright is not a declared application dependency. Passing tests verify the exercised behavior, rather than proving every engineering calculation, provider integration or production load scenario.

GitHub Actions runs repository CI. Docker uses a multi-stage Node 22 Debian Bookworm slim image, compiles both workspaces and runs the final application as an unprivileged user. Docker Compose and a Render blueprint provide deployment recipes. Production preflight scripts check required configuration and built entry points. Production frontend source maps are disabled.

The complete application requires a running backend. A static host can serve frontend assets only when API routing and the authoritative backend are separately provided. HTTPS, stable encryption keys, private database and provider credentials, persistent scratch storage where needed, backup restoration and monitoring remain deployment responsibilities.

Local commands

From the repository root, use Node 22.12.0 or newer and the checked-in lockfile. Configure private environment files from the examples before using account, mail or database workflows.

```
npm ci
npm run dev
```

Useful verification and production commands are:

```
npm run lint
npm test
npm run test:postgres-flow
npm run build
npm run check:build
npm start
```

Database migrations and imports are explicit operations. Follow the Supabase runbook rather than running them against an existing database without checking the target and backup. See the deployment guide, Supabase guide and Gmail OTP guide for environment-specific instructions. Never put secrets in browser-visible `VITE_` variables or in documentation.

8 Current environment and source guide

Environment status on 20 October 2026

The local application has been connected to Supabase PostgreSQL and existing records migrated with their identifiers and encrypted values preserved. A SQLite backup was retained. The latest database cutover verification covered login, encrypted project save and reload, session use and logout, with temporary test data removed afterward. The associated backend suite passed 105 tests and the production build passed.

The frontend and API remain local development services; a public application deployment has not been completed. Automatic cloud backups still need configuration. Provider-specific live acceptance, full numerical engineering validation and disabled native execution remain separate work. These status statements describe the recorded environment and checks on this date, rather than a continuing uptime or security guarantee.

Where the implementation lives

Repository area	Responsibility
`client/studio.html` and `client/studio/`	Public site content, scenes, tokens, interactions and component styles.
`client/src/App.tsx` and `client/src/pages/`	React routes, dashboard, editors, technical workbenches and account screens.
`client/src/editor/` and `client/src/lib/`	Three.js editing, domain modules, exchange, computation and browser persistence.
`client/build/` and `client/public/sw.js`	Build inventory and offline shell behavior.
`server/src/routes/`	HTTP endpoints for authentication, projects, collaboration, workspaces and integrations.
`server/src/security.ts` and `server/src/crypto.ts`	Sessions, CSRF, throttling and encryption primitives.
`server/src/db.ts` and database adapters	Database abstraction, transaction ownership and PostgreSQL or SQLite selection.
`server/supabase/migrations/` and `server/certs/`	Explicit schema migrations and the public certificate trust material.

Repository area	Responsibility
`server/test/`, `client/test/` and `scripts/`	Automated tests, mail setup, deployment checks and database verification tooling.
`Dockerfile`, `docker-compose.yml`, `render.yaml` and `.github/workflows/`	Container, local orchestration, hosting blueprint and CI configuration.
Workspace manifests and `package-lock.json`	Authoritative direct dependency declarations and resolved dependency graph.

Further detail is available in the studio guide, technical workspace guide, engineering guide, 3D frame scope, BIM and civil guide, offline shell guide and native execution boundaries. Older specifications describe earlier plans and should be read alongside current code and manifests.